Predicting High Income from the 1994 US Census
Supervised Learning Project



Naye Kanouté/ Nicolas Dollet/ Danila ZHEKHOVSKII

20/10/2026

Dataset: Adult / Census Income (UCI Machine Learning Repository, loaded via OpenML, version 2)
Binary classification: annual income `>50K` vs `<=50K`

We predict whether an individual earns more than 50,000k a year from 14 census attributes. After cleaning and engineering six features, we compare three models of increasing complexity (logistic regression, random forest, XGBoost), all wrapped in a single scikit−learn Pipeline, tuned with Optuna under stratified 5−fold cross−validation and tracked with MLflow.

We show that class re−weighting only shifts the decision threshold while distorting probabilities, and therefore handle the imbalance through threshold selection instead. The final model, an unweighted tuned XGBoost, reaches a PR−AUC of 0.827 in cross−validation and **0.813 on the held−out test set**, is well calibrated (ECE = 0.004), and achieves a recall of 0.79 at a cost−motivated threshold of 0.35.

Problem framing and protocol

Data. The Adult dataset was extracted from the 1994 US Census by Barry Becker and published by Kohavi (1996). It contains 48,842 rows; one row is one person, described by age, work class, education, marital status, occupation, relationship in the household, race, sex, capital gains and losses, weekly working hours and native country.

Task and metric. The target is 1 if annual income exceeds 50K (1994 dollars). Only **23.9%** of individuals are positive, so a classifier that always predicts `<=50K` already reaches 76.1% accuracy with a recall of 0 (we verified this with a `DummyClassifier`). Accuracy is therefore uninformative. Our **primary metric is PR-AUC** (average precision): it summarises how well the rare class is ranked, independently of any threshold. Precision, recall and F1 are reported for decisions; ROC-AUC, Brier score and ECE complete the picture.

Protocol. A stratified **60/20/20** split (`random_state=42`) gives a train set (29,274 rows), a validation set (9,758) and a test set (9,758), each with 23.9% positives. All modelling choices are made on the train set (via cross-validation) and the validation set. The test set is used **once**, at the very end. For calibration and threshold selection, the validation set is further split into two stratified halves.

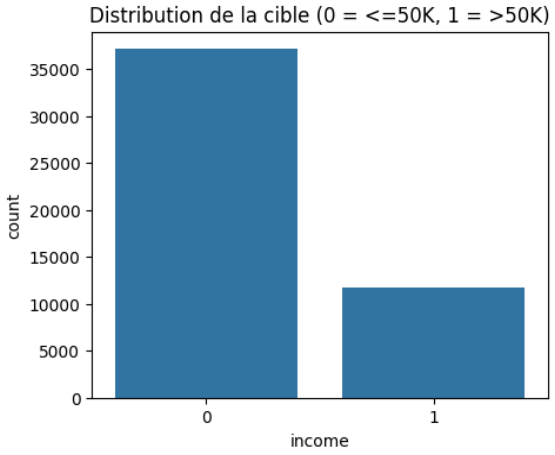
Exploratory analysis and cleaning

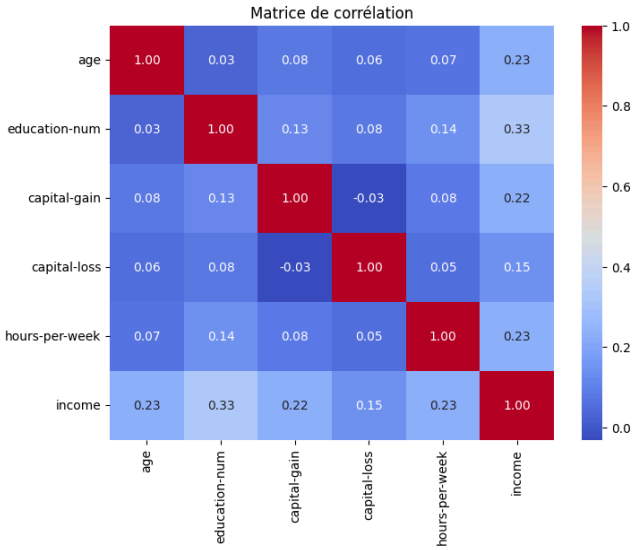
Missing values. Only three categorical columns are affected: `workclass` (2,799, 5.7%), `occupation` (2,809, 5.8%) and `native-country` (857, 1.8%). In OpenML version 2 they are already encoded as `NaN`. Rather than dropping 7% of the rows, we impute the most frequent category *inside the Pipeline*, so the imputation statistics are learned on the training fold only.

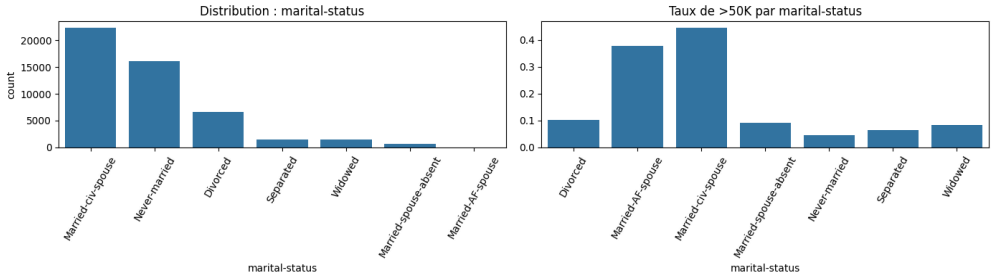
Duplicates. The raw data contain 52 exact duplicates, which we remove (48,790 rows left). The order matters: searching for duplicates *after* dropping `fnlwgt` flags of more than 6,000 rows, which are in fact distinct people sharing a common profile. Removing them would have discarded 13% of the data and shifted the positive rate from 23.9% to 24.6%.

Dropped columns. `fnlwgt` is a census sampling weight, not a property of the person. `education` is redundant with `education-num`: we checked that each category maps to exactly one code, and keep the ordinal version (1 = Preschool, 9 = HS-grad, 13 = Bachelors, 16 = Doctorate).

Distributions. Age ranges from 17 to 90 (mean 38.7). `education-num` has a median of 10 (“Some-college”). `capital-gain` and `capital-loss` are zero for more than 75% of people, with extreme values up to 99,999 (a census cap). Working hours peak sharply at 40 (median and first quartile). Correlations between numerical features are weak (|r|<0.2), so multicollinearity is not a concern; the very different scales motivate standardisation for the linear model.







Target distribution (23.9% positives). Correlation matrix of numerical features. Distribution of `marital-status` *and positive rate per category, the strongest single signal in the data.*

Categorical variables. 91.4% of people were born in the US, and the remaining 40 countries are each below 2%. `workclass` has two near-empty categories (Without-pay, Never-worked). Married people have a much higher positive rate than all other marital statuses. `sex` (66.8% male) and `race` (85.5% White) are highly unbalanced, and their positive rates differ markedly by group: these are the sensitive attributes of the dataset.

Feature engineering

Each new feature is justified by the gap it creates in the positive rate (base rate 23.9%):

Engineered features and their discriminative power (computed on the full cleaned dataset).

Feature	Definition	Positive rate
`is_married`	Married-civ-spouse or Married-AF-spouse	44.6% (married) vs 6.4%
`has_capital`	capital-gain >0 or capital-loss >0	57.5% vs 18.9%
`net_capital`	capital-gain − capital-loss	–
`hours_group`	Part-time (<35 h), Full-time (35–40 h), Overtime (>40 h)	6.9% / 20.6% / 40.0%
`workclass` (regrouped)	Private, Self-employed, Government, Other	21.8 / 36.3 / 30.8 / 6.5%
`is_us`	born in the United States	24.4% vs 18.7%

`is_married` condenses the seven marital categories into the one contrast that matters (a 7× ratio). `has_capital` is more robust than the heavily skewed raw amount. `hours_group` captures a step-wise effect that a linear term on raw hours would miss. Regrouping `workclass` (8 → 4 levels) and replacing `native-country` by `is_us` (41 levels → 1 column) remove rare levels that would otherwise produce near-empty one-hot columns. We note that `net_capital` is an exact linear combination of two existing columns: harmless for trees, but it makes the logistic regression coefficients non-identifiable (the L2 penalty keeps the fit stable). The original `marital-status` and `native-country` columns are dropped after transformation.

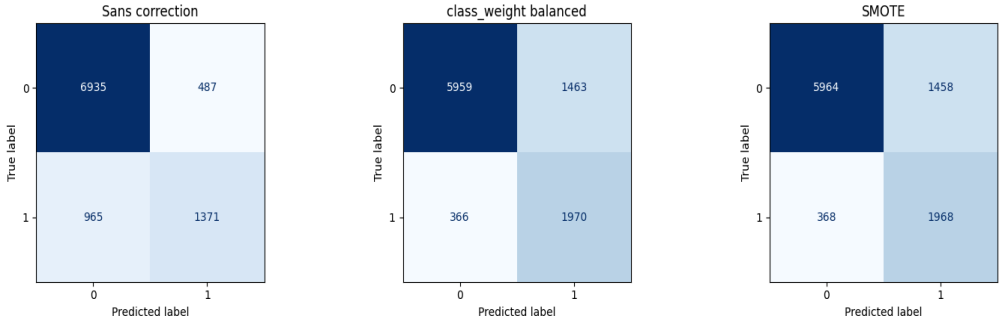
All of this is implemented as a single function `add_features`, applied inside the Pipeline.

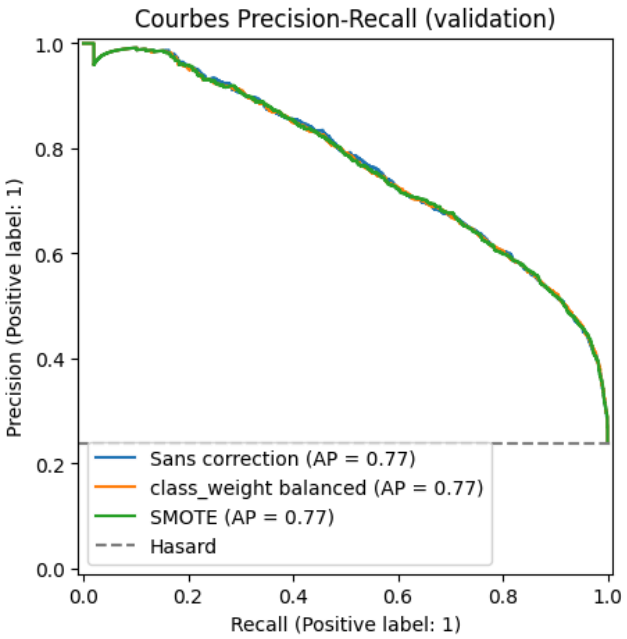
Class imbalance

We compare three treatments on a logistic regression, evaluated on the validation set: no correction, `class_weight="balanced"` (weights 0.66 for class 0 and 2.09 for class 1, i.e. a positive error costs about 3× more), and SMOTE (synthetic positives generated only during `fit`, using an `imblearn` Pipeline).

Effect of imbalance treatments (logistic regression, validation set, threshold 0.5).

Treatment	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
None	0.851	0.738	0.587	0.654	0.772	0.907
`class_weight="balanced"`	0.813	0.574	0.843	0.683	0.770	0.907
SMOTE	0.813	0.574	0.842	0.683	0.770	0.907





Confusion matrices and precision-recall curves of the three treatments. The PR curves are superimposed.

Interpretation. The three models have the *same* PR-AUC (0.770–0.772) and the *same* ROC-AUC (0.907), and their PR curves overlap: they rank individuals identically. Re-weighting and SMOTE do not produce a better model; they only move the operating point along the same curve (recall 0.59 → 0.84, precision 0.74 → 0.57). Moving the operating point can be done directly by choosing the decision threshold, without changing the

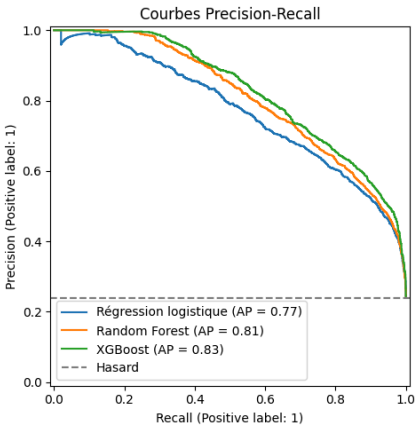
training distribution. Re-weighting also has a side effect: the model is trained on an artificial class balance, so its predicted probabilities are inflated.

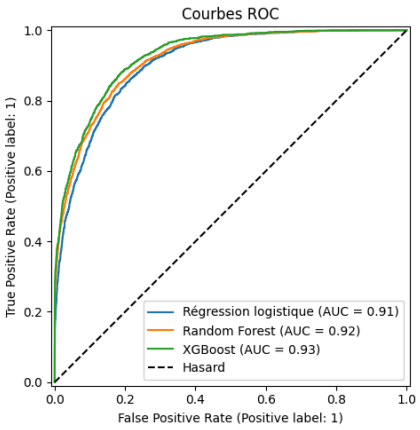
Decision. We therefore **do not re-weight**. Imbalance is handled by imbalance-aware metrics (PR-AUC for model selection) and by an explicit, cost-motivated **decision threshold**.All subsequent models are trained without `class_weight` or `scale_pos_weight`.

Modelling progression

Three models of increasing complexity, same preprocessing, validation set (threshold 0.5).

Model (defaults)	Acc.	Prec.	Recall	F1	PR-AUC	ROC-AUC	Fit time
Logistic regression	0.851	0.738	0.587	0.654	0.772	0.907	1.3 s
Random forest (300 trees)	0.863	**0.778**	0.601	0.678	0.809	0.918	15.0 s
XGBoost (300 trees, depth 6)	**0.870**	0.768	**0.658**	**0.708**	**0.826**	**0.927**	8.5 s





Precision-recall and ROC curves of the three models on the validation set.

What each step bought us.

- **Logistic regression → random forest** (+3.7 PR-AUC points): trees capture non-linear effects and interactions that a linear model cannot represent, for example the effect of education depending on age and marital status. The price is interpretability and an 11× longer fit.
- **Random forest → XGBoost** (+1.7 points): boosting fits each tree to the errors of the previous ones. It improves every metric, including recall at 0.5, and trains faster than the forest.
- **Diminishing returns**: most of the gain comes from moving to trees; the extra step from bagging to boosting is smaller

Pipeline

Preprocessing and model live in a single scikit-learn `Pipeline`:

```
Pipeline([
  ("features", FunctionTransformer(add_features)),          # feature
engineering
  ("columns", ColumnTransformer([
     ("num", Pipeline([SimpleImputer("median"), StandardScaler()]),
NUM_COLS),
     ("cat", Pipeline([SimpleImputer("most_frequent"),
                       OneHotEncoder(handle_unknown="ignore")]),
CAT_COLS)])),
  ("model", XGBClassifier(...)),
])
```

- **No leakage**: medians, modes, means, standard deviations and one-hot categories are learned in `fit` on training data only, and inside each CV fold during tuning. One-hot encoding is done here rather than with `pd.get_dummies` on the full data frame for the same reason; `handle_unknown="ignore"` also protects against categories unseen in training.
- **Raw input**: the Pipeline takes rows exactly as they appear in the dataset (strings, NaN) and outputs a probability. We checked this on five raw validation rows.
- **One object** is tuned, logged, explained, calibrated and evaluated, which keeps every result consistent. After preprocessing, the model sees 43 columns.

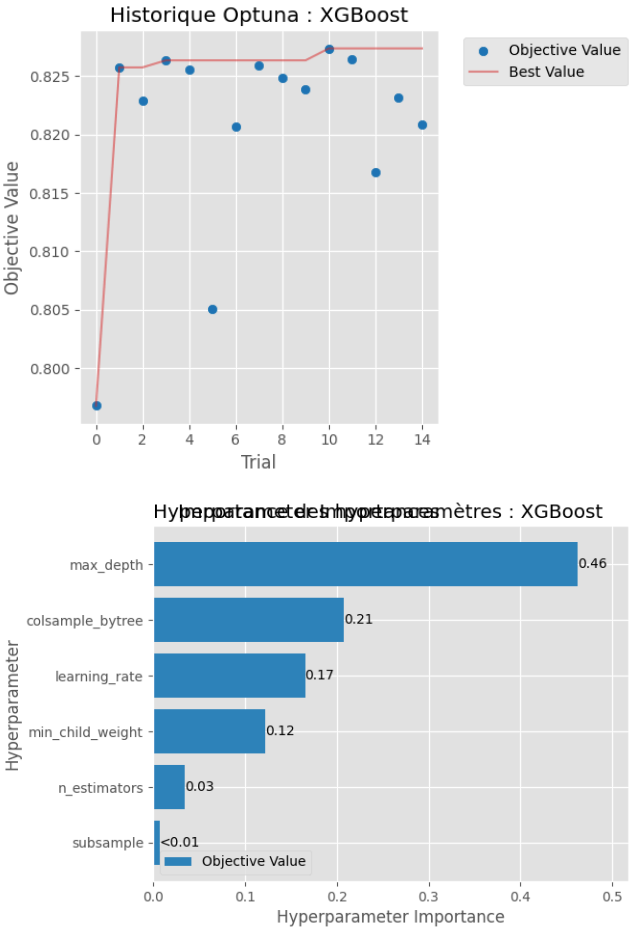
For SMOTE, `imblearn` does not accept nested Pipelines, so the two preprocessing steps are inserted directly into the `imblearn` Pipeline.

Hyperparameter tuning with Optuna

Method. For each model, Optuna (TPE sampler, seed 42) runs 15 trials. Each trial is scored by **stratified 5-fold cross-validation on the training set only**, and the objective is the mean PR-AUC over the folds. The validation set is never seen during the search, so it provides an independent check of the CV estimate.

PR-AUC with default vs tuned hyperparameters, compared under the same 5-fold CV. Best XGBoost: 250 trees, learning rate 0.030, depth 8, subsample 0.87, colsample 0.76, `min_child_weight` 2.

Model	CV default	CV tuned	Gain (CV)	Validation tuned	Val. − CV
Logistic regression	0.7781	0.7784	+0.0002	0.7720	−0.0063
Random forest	0.8122	0.8143	+0.0021	0.8102	−0.0041
XGBoost	0.8265	**0.8274**	+0.0009	**0.8240**	−0.0034



Optuna search for XGBoost: optimisation history and hyperparameter importance.

Analysis:

- **The CV score is not optimistic**: the validation score which Optuna never saw is within 0.007 of the CV score for all three models.
- **Tuning gains are negligible** (+0.0002 to +0.0021), well within the fold-to-fold standard deviation (≈0.006).
- For XGBoost, `max_depth` is by far the most influential hyperparameter (importance 0.46), followed by `colsample_bytree` and the learning rate.
- The ranking XGBoost > random forest > logistic regression is stable across CV and validation. **We select the tuned XGBoost** on its CV score, which is the most reliable estimate because it averages over five folds. This single model is used in all following sections.

Experiment tracking

Every Optuna trial is logged as an MLflow run (SQLite backend, experiment `adult-income`):

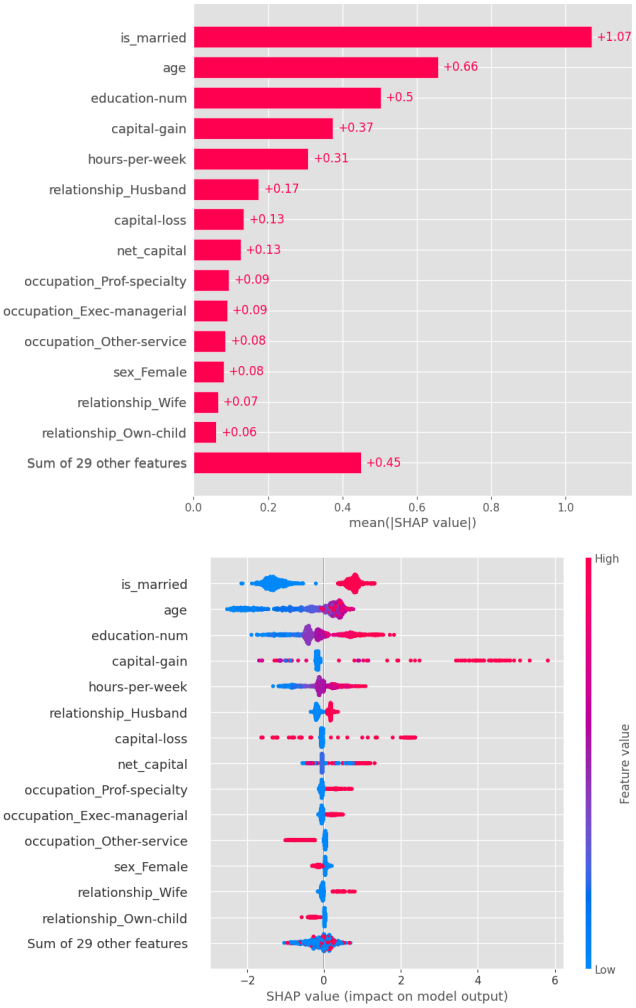
- **Structure**: one parent run per model family, with 15 nested child runs (45 trials in total);
- **Parameters**: the hyperparameters of each trial, plus the best parameters in the parent run;
- **Metrics**: mean and standard deviation of the CV PR-AUC and fit duration for each trial; validation accuracy, precision, recall, F1, PR-AUC and ROC-AUC for the best model;

The best XGBoost trial (trial 10) reaches a CV PR-AUC of 0.8274 with a fold standard deviation of 0.0059; the top ten trials are all XGBoost runs, between 0.8229 and 0.8274. One practical issue: recent MLflow versions

save scikit-learn models with `skops` by default, which rejects our custom `add_features` function, so we use `serialization_format="cloudpickle"`.

Explainability using SHAP

We apply `shap.TreeExplainer` to the tuned XGBoost, on 1,000 validation rows transformed by the Pipeline. Values are in log-odds; the base value E[f(X)] = −1.165 corresponds to the average predicted probability (≈24%). Feature values on the plots are standardised.



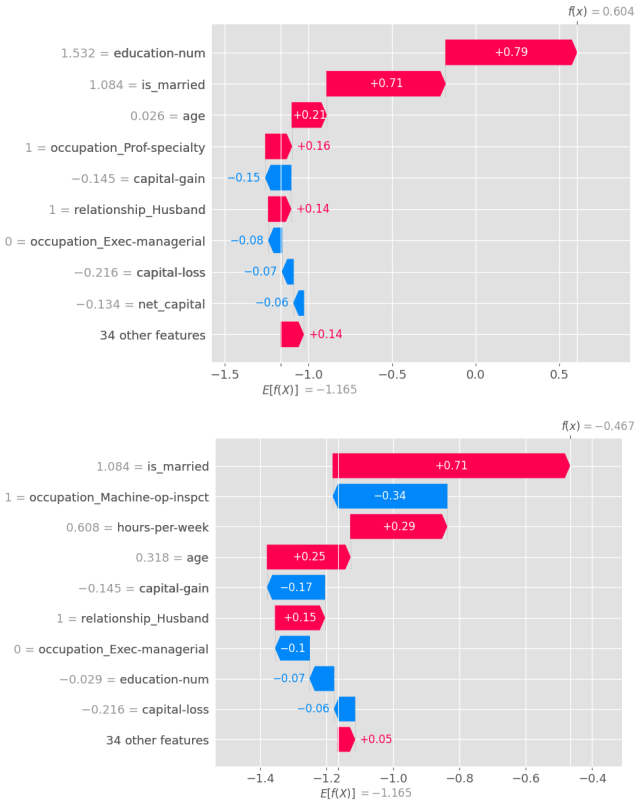
Global importance and beeswarm plot

Global importance. The ranking is `is_married` (1.07), `age` (0.66), `education-num` (0.50), `capital-gain` (0.37), `hours-per-week` (0.31), then `relationship_Husband` (0.17) and `capital-loss` (0.13). The beeswarm plot shows monotone, intuitive directions: being married, older, more educated and working longer hours all push towards `>50K`. Capital gains have a rare but very strong effect (up to +6 log-odds). Occupations split into high-income (Prof-specialty, Exec-managerial) and low-income (Other-service) groups.

Local explanations. We explain one prediction of each type:

- **True positive** (f(x) = 1.90, p≈0.87): 45-year-old married man with a Master's degree, Exec-managerial, 50 h/week. Education (+1.01), marriage (+0.67), occupation (+0.46), hours (+0.44) and age (+0.36) all point the same way.
- **True negative** (f(x) = −5.66, p≈0.003): 24-year-old single man, HS-grad, Handlers-cleaners. Young age (−1.61), not married (−1.34), occupation (−0.48) and education (−0.43) all agree.

- **False positive** (f(x) = 0.60, p≈0.65): 39-year-old married man, Master's, Prof-specialty in local government, 72 h/week. The profile looks like a high earner (education +0.79, marriage +0.71), but he earns `<=50K`. A plausible explanation is that public-sector pay scales are compressed, which the model only partly captures through the regrouped `workclass`.
- **False negative** (f(x) = −0.47, p≈0.39): 43-year-old married man, some college, machine operator, 48 h/week. Marriage (+0.71), hours (+0.29) and age (+0.25) push up, but the manual occupation (−0.34) pulls him below 0.5. At our chosen threshold of 0.35 he would be correctly classified as `>50K`: a concrete example of the value of threshold tuning.



SHAP waterfall plots of the false positive and the false negative.

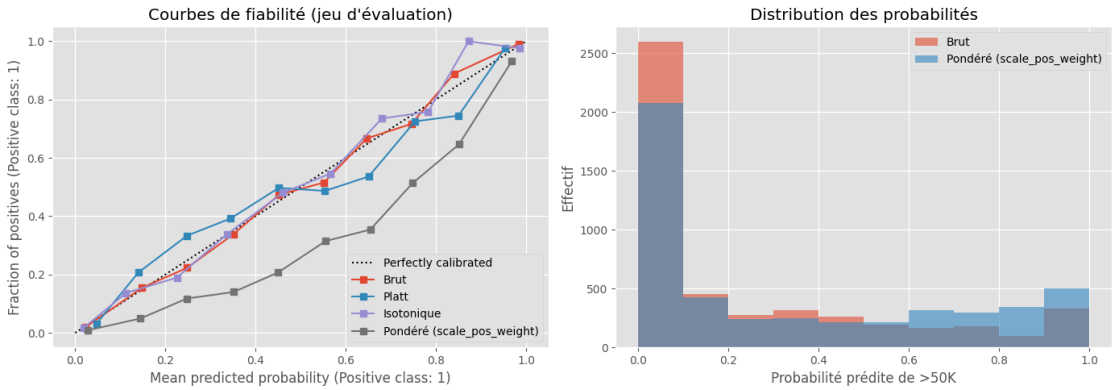
Business sense. The drivers are plausible. Marriage proxies age, career stability and household specialisation; education and occupation proxy qualification; hours and capital proxy labour supply and wealth. Two caveats matter for deployment. First, `relationship_Husband` carries a positive effect beyond `is_married`: since a husband is by definition a man, this is an **indirect channel for sex**. Second, `sex_Female` itself appears in the top 15 with a negative contribution. The model reproduces the 1994 gender pay gap present in the data. We did not run a formal fairness audit; this is our main limitation.

Calibration

Method. Following the course approach, the tuned XGBoost (fitted on the training set) is **frozen** with `FrozenEstimator`, and calibrators are fitted on a separate **calibration set**, half of the validation set (4,879 rows). Calibration quality is measured on the other half, the **evaluation set** (4,879 rows). We compare the raw model, Platt scaling (sigmoid) and isotonic regression. We add the same XGBoost trained *with* `scale_pos_weight` to quantify the cost of the re-weighting rejected in Section 4. The ECE uses 10 equal-width bins assigned with `np.digitize`, so probabilities of exactly 0 or 1 are counted.

Calibration on the evaluation set (true positive rate: 23.9%).

Model (evaluation set)	Brier	ECE	Mean predicted p	PR-AUC
Raw XGBoost (unweighted)	**0.0872**	**0.0107**	24.2%	**0.831**
Platt scaling	0.0893	0.0347	24.4%	0.831
Isotonic regression	0.0877	0.0119	24.4%	0.813
XGBoost with `scale_pos_weight`	0.1093	0.1073	34.7%	0.831



Reliability diagram (left) and distribution of predicted probabilities for the raw and re-weighted models (right).

Reliability per bin for the raw XGBoost (evaluation set).

Bin	0–.1	.1–.2	.2–.3	.3–.4	.4–.5	.5–.6	.6–.7	.7–.8	.8–.9	.9–1
Count	2597	454	278	320	264	194	165	180	99	328
Mean predicted	.023	.148	.248	.351	.452	.552	.647	.746	.840	.983
Observed rate	.020	.154	.223	.338	.473	.515	.667	.717	.889	.991

- **The re-weighted model is clearly over-confident**: its curve lies below the diagonal everywhere. It predicts 34.7% positives on average against 23.9% observed (ECE = 0.107). For instance, among people it scores around 0.55, only about 30% earn `>50K`. This is a direct consequence of `scale_pos_weight` and confirms the decision of Section 4.
- **The raw, unweighted XGBoost is already well calibrated**: ECE = 0.011, Brier = 0.087 (against ≈0.182 for a constant prediction of 23.9%), and a mean predicted probability of 24.2%. Per bin, the gap between predicted and observed rates is small, at most 0.05 (in the 0.8–0.9 bin). It is slightly *under*-confident there, but that bin holds only 99 people, so its observed rate is the least precise. Most predictions sit in the extreme bins (2,597 below 0.1 and 328 above 0.9), where calibration is best.
- **Re-calibration does not help.** Platt scaling worsens the ECE (0.035): the miscalibration is not sigmoid-shaped, so a two-parameter sigmoid distorts an already good mapping. Isotonic regression is neutral on ECE but lowers the PR-AUC (0.813), because its step function creates ties that destroy part of the ranking. With 4,879 calibration rows, the extra flexibility costs more than it gains.

The probabilities can feed a downstream decision directly (expected cost, risk scores, forecasting the number of high-income clients). The re-weighted variant could not be used this way without re-calibration.

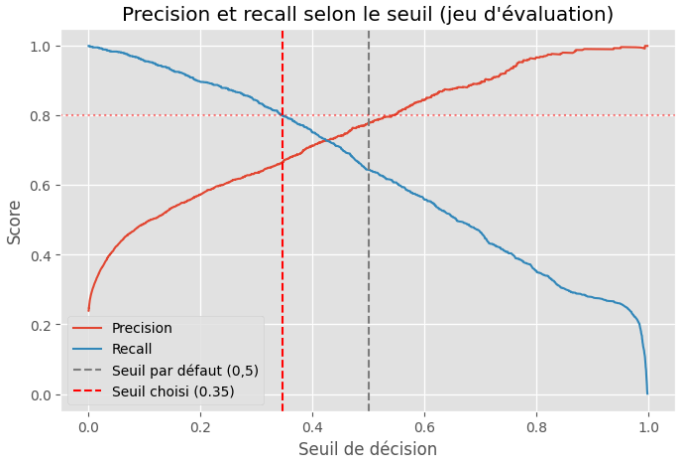
Final model: raw XGBoost, selected automatically as the lowest Brier score among the raw, Platt and isotonic versions.

Decision threshold

Deployment scenario. A bank uses the model to target a premium offer at high-income clients. A **false negative** (missing a >50K client) loses a profitable customer; a **false positive** costs one unnecessary mailing or call. **Missing a high-income client costs more than contacting a low-income one**, so we favour recall. We set a **target recall of 80%** and choose, with `precision_recall_curve` on the evaluation set, the threshold that maximises precision under this constraint.

Default vs cost-motivated threshold.

Threshold (evaluation set)	Value	Precision	Recall	F1	Predicted >50K
Default	0.500	0.778	0.644	0.705	966
Recall ≥ 80%	**0.346**	0.665	**0.801**	**0.727**	1,407



Precision and recall as a function of the threshold; the dashed red line marks the chosen threshold.

At 0.5 the model misses 36% of high earners. Lowering the threshold to 0.346 raises recall to 80% at the cost of 11 precision points: the bank contacts 441 more people (1,407 instead of 966) but misses far fewer valuable clients, and F1 even improves. Because the probabilities are calibrated, the threshold has a direct meaning: we contact anyone with at least a 35% chance of being a high earner. **Changing the threshold is the least invasive intervention**: the model is not retrained, only the decision rule changes, and the rule can be revised if the business costs change (a costly contact channel would call for a higher threshold).

Final evaluation on the test set

Every choice was made without the test set: features, imbalance treatment, model and hyperparameters (train set and CV), calibration method (calibration set) and threshold (evaluation set). We evaluate the final model **once** on the 9,758 test rows. We deliberately do not refit on train + validation: the calibration and threshold decisions were validated for this specific fitted model, and refitting would leave no independent data to re-check them.

Final, one-shot evaluation of the tuned, unweighted XGBoost on the test set.

Test set	PR-AUC	ROC-AUC	Precision	Recall	F1	Brier	ECE
Threshold 0.50	0.813	0.923	0.764	0.635	0.694	0.092	0.004
Threshold 0.35 (chosen)	0.813	0.923	0.652	**0.791**	**0.715**	0.092	0.004

- **Generalisation.** The test PR-AUC (0.813) is 0.014 below the CV estimate (0.827) and 0.011 below validation (0.824), and the ROC-AUC (0.923) is close to validation (0.926). This small drop is the usual price of having selected the model on CV and validation scores, and the CV estimate remains a good guide.
- **Calibration holds**: ECE = 0.004 and Brier = 0.092 on unseen data.
- **The decision rule holds**: at threshold 0.35, the recall is 0.79 (target 0.80) with a precision of 0.65, nearly identical to the evaluation set.

Limitations and next steps

- **Fairness was not audited.** SHAP shows that sex affects predictions directly (`sex_Female`) and indirectly (`relationship_Husband`). We kept `sex` and `race` on purpose: dropping them would not remove the bias, because proxies remain, and it would make the bias impossible to measure. A deployment would require demographic-parity and equalized-odds measurements by sex and race, and possibly a mitigation such as group-specific thresholds (`ThresholdOptimizer`).
- **Dated and narrow data**: 1994 incomes, a nominal $50K threshold that is not inflation-adjusted, and a US-only population.
- **Missing information**: sector, seniority, region and non-wage income are absent, which explains several errors (e.g. the public-sector false positive).
- **Serving**: the Pipeline already accepts raw rows and can be exposed through an API endpoint; the notebook code still has to be exported into `.py` modules.
- **Tuning budget**: 15 trials per model is modest, but the flat optimisation landscape suggests little is left to gain.

Conclusion

A single leakage-free Pipeline, from raw census rows to a decision, reaches a PR-AUC of 0.827 in cross-validation and 0.813 on the test set with XGBoost. The modelling progression shows a gain from linear to tree models and diminishing returns afterwards, and Optuna confirms that the remaining ceiling comes from the data. Our main methodological lesson concerns class imbalance: re-weighting does not improve ranking but inflates probabilities (ECE 0.107 vs 0.011), whereas an unweighted model with an explicit, cost-motivated threshold gives both calibrated probabilities and the desired recall (0.79 on the test set). SHAP explanations make business sense but reveal a dependence on sex, which a real deployment would have to audit.

Use of AI

An AI assistant (Claude, Anthropic) was used to help the code skeletons (Pipeline, Optuna/MLflow integration, SHAP, calibration), to debug errors (nested Pipeline in `imblearn`, MLflow model serialisation), and to draft a v1 of this report. All results come from our own runs; the methodological choices and interpretations were reviewed and validated by the group.

Contributions

Member	Contributions
Naye KANOUTE	EDA, cleaning, feature engineering, report sections 1–3
Nicolas DOLLET	Imbalance study, models, Pipeline, Optuna, MLflow
Danila ZHEKHOVSKII	SHAP, calibration, threshold, test evaluation, README